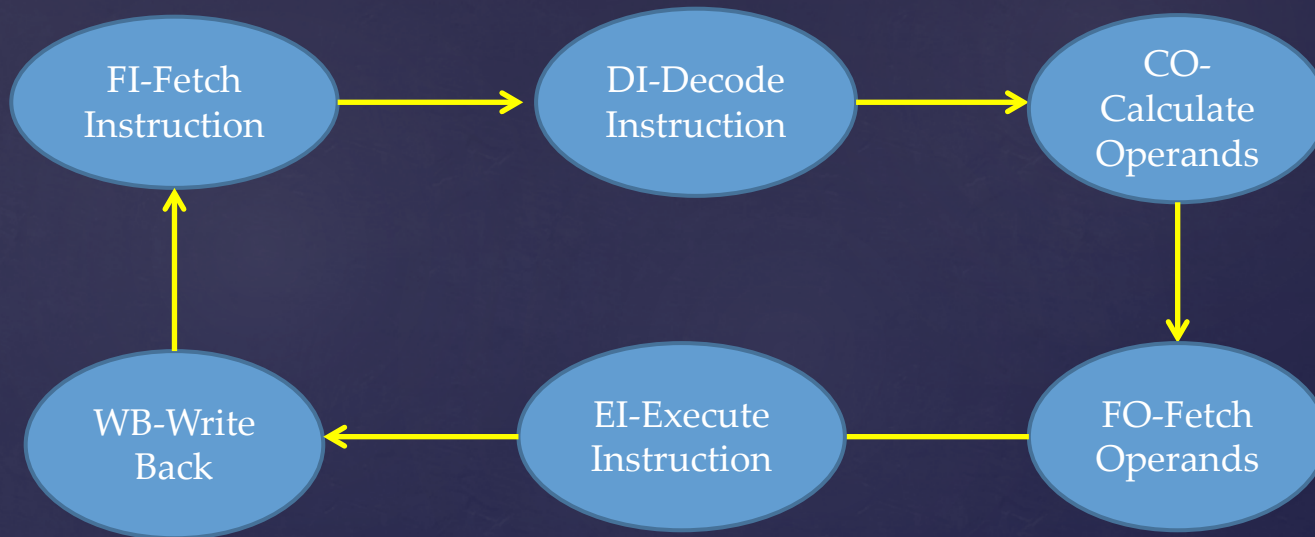


Introduction to Pipelining

This is the most general possible CPU control logic:



In a well-designed datapath, we can arrange matters so each of these steps requires **one clock cycle**. In time it would look like this:

Clock cycle	0	1	2	3	4	5	6	7	8	9	10	12	13	14
I1	FI	DI	CO	FO	EI	WO								
I2							FI	DI	CO	FO	EI	WO		
I3													FI	DI
...														

Drawn like this, it looks like we ought to be able to do better.

Suppose we arrange the datapath like an “assembly line”. We assume we have one “work station” in the CPU that performs each step. We can compress execution like this:

[illegible]

Pipelining

We call this idea **pipelining** and it is a major component of modern high-performance CPU design.

- In principle, we will see **one instruction executed per cycle**. That is just good.
- If a specific instruction doesn't need some step, that's not a problem. Just **don't do it**. Also good.
- Less good are some issues we can illustrate on the table

Consider the situation once the pipeline is full:

Clock cycle	0	1	2	3	4	5	6	7	8	9	10	12	13	14
I1	FI	DI	CO	FO	EI	WO								
I2		FI	DI	CO	FO	EI	WO							
I3			FI	DI	CO	FO	EI	WO						
I4				EI	DI	CO	FO	EI	WO					
I5					FI	DI	CO	FO	EI	WO				
I6						FI	DI	CO	FO	EI	WO			
I7							FI	DI	CO	FO	EI	WO		
...														

These three all need arithmetic

These three all need memory access

- At steady state, we will have three instructions that all do arithmetic. **NO PROBLEM**. Solution: Build more hardware
- At steady state, we could have three instructions that all need memory access. **PROBLEM**. Partial solutions:
 - Split I-cache & D-cache
 - **Don't allow** multiple accesses in one instruction (**LD/ST** only)
- Less easy to understand is a problem illustrated on the next page:

This is called a **dependency**. We will talk more about them later:

Clock cycle	0	1	2	3	4	5	6	7	8	9	10	12	13	14
I1	FI	DI	CO	FO	EI	WO								
I2		FI	DI	CO	FO	EI	WO							
I3			FI	DI	CO	FO	EI	WO						
I4				FI	DI	CO	FO	EI	WO					
I5					FI	DI	CO	FO	EI	WO				
I6						FI	DI	CO	FO	EI	WO			
I7							FI	DI	CO	FO	EI	WO		
...														

What if I1 computes the address needed in I2?

I need it here

But I compute it here

One more problem. Suppose I2 is a conditional branch to I16:

Clock cycle	0	1	2	3	4	5	6	7	8	9	10	12	13	14
I1	FI	DI	CO	FO	EI	WO								
I2		FI	DI	CO	FO	EI	WO							
I3			FI	DI	CO	FO	EI	WO						
I4				FI	DI	CO	FO	EI	WO					
I5					FI	DI	CO	FO	EI	WO				
I16						FI	DI	CO	FO	EI	WO			
I17							FI	DI	CO	FO	EI	WO		
...														

Here is where I know I should branch

This is the first instruction after the branch

Everything in red DIDN'T HAPPEN

This is called the **branch penalty**. We will discuss solutions in class.